

2025年全国大学生计算机系统能力大赛——
智能计算创新设计赛（先导杯）
南开大学 校内赛
培训

2025年5月





(二) 赛事安排



赛事主题

“智能计算，
智享未来”



参赛对象

- 南开大学在校本科生
- 个人形式参赛



赛程安排

时间：5月14日- 6月4日

赛题：

- 1) 基础题（课程综合大作业作业，必做）；
- 2) 进阶题，共2道

提交时间：6月4日24:00前

提交邮箱：

nkucs_org_2025s@163.com



奖项设置

特等奖（10名）：

奖金**1000元**、荣誉证书

一等奖（40名）：荣誉证书

奖金**200元**、荣誉证书

二等奖（70名）：荣誉证书

优胜奖（若干）：荣誉证书

“先导杯”

南开大学 校内赛





(二) 赛事安排

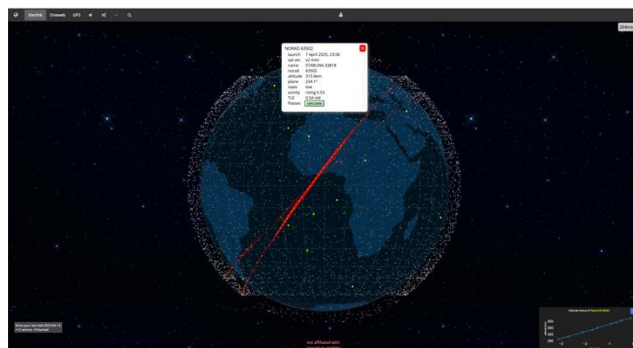
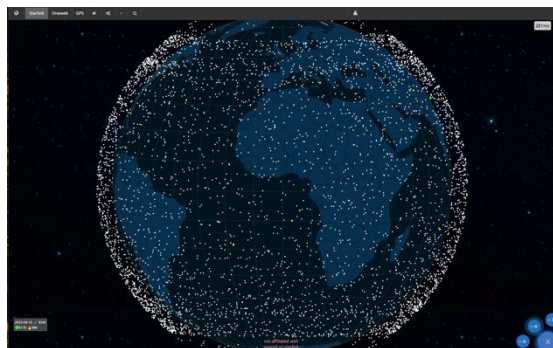
“先导杯” 南开大学 校内赛评分规则



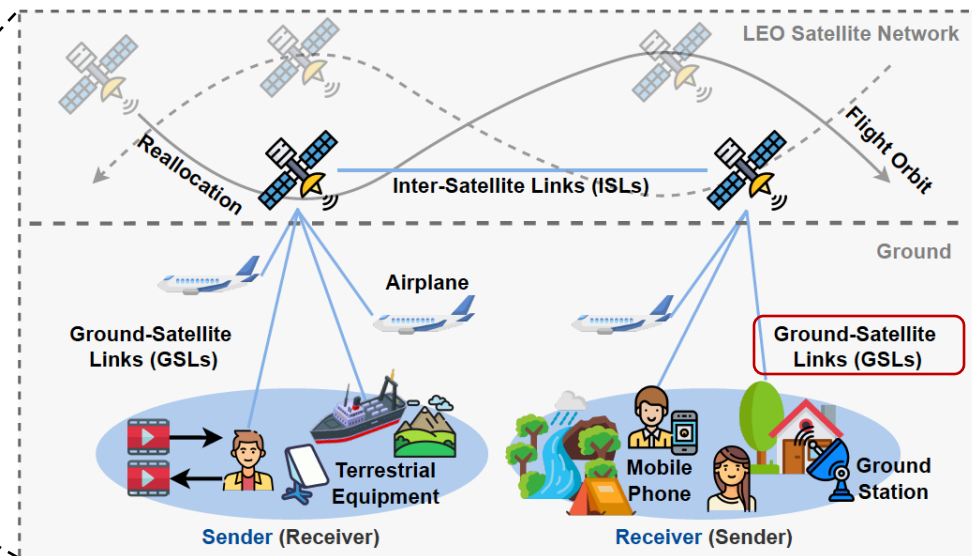


(三) 赛题设置

赛题背景:



SpaceX Starlink 低轨卫星网络星座



基于低轨卫星的网络服务

低轨 (LEO) 卫星网络^[1, 2]因其低时延、高覆盖的优势, 正成为未来全球广域网络服务的重要补充。目前, **SpaceX、OneWeb**等公司已部署**数千颗卫星**, 初步形成**星座网络**; 我国**星网工程**也在加快推进, 积极构建**天地一体化信息网络**。LEO卫星网络具备动态拓扑、链路多变、频繁切换等特点, 使其网络服务面临带宽波动性大、链路预测难等挑战。因此, 提升服务质量的关键之一在于**精准的网络带宽预测**。借助机器学习模型, 可实现对历史网络状态的深度建模与未来网络带宽的有效预测, 但如何实现**高效且实时**的预测, 要求对**机器学习的计算过程进行深度优化**。

^[1] <https://satellitemap.space/> ^[2] <https://www.bilibili.com/video/BV1nm42137eG>



(三) 赛题设置

机器计算学习过程的核心计算单元是**矩阵乘法运算**。在实际应用中，如何高效利用**加速硬件**（如曙光DCU，英伟达GPU等）和**并行计算算法**完成**大规模矩阵乘**，成为智能计算系统设计的关键问题。为应对高效准确LEO卫星带宽预测挑战，本次校内赛将**围绕基于矩阵乘法的多层感知机（MLP）神经网络计算优化**展开，通过设计一系列挑战任务，培训并引导参赛者从算法理解、性能建模、系统优化到异构调度完成一个完整的系统创新设计。

编程语言：C++，DTK； 环境：曙光DCU实训平台； 算力：8核CPU+16G内存+1张DCU加速卡

基础题：智能矩阵乘法优化挑战（必做）



已知两个矩阵：矩阵 A（大小 $N \times M$ ），矩阵 B（大小 $M \times P$ ）：

- ❑ **问题一**：请完成标准的矩阵乘算法，并支持浮点型输入，输出矩阵为 $C = A \times B$ ，并对随机生成的浮点数矩阵输入，验证输出是否正确（ $N=1024$ ， $M=2048$ ， $P=512$ ）；
- ❑ **问题二**：请采用至少两种方法加速以上矩阵运算，鼓励采用多种优化方法和混合优化方法；理论分析优化算法的性能提升，并可通过hipprof、hipgdb、rocm-smi等工具进行性能分析和检测；



(三) 赛题设置

基础题：智能矩阵乘法优化挑战——基本思路

➤ 多线程并行化加速

利用多核CPU计算资源、可采用OpenMP (Open Multi-Processing) 优化

➤ 子块并行优化

通过局部计算降低内存访问延迟；为OpenMP或其他并行机制提供更细粒度、更均匀的工作划分，适用于大规模矩阵，特别适合在多核CPU上运行

➤ 多进程并行优化

使用MPI (Message Passing Interface) 实现矩阵乘法的多进程优化，其核心思想是将大矩阵按行或块划分给不同进程，利用进程间通信协同完成计算

➤ DCU加速计算

通过曙光DCU (Dawn Computing Unit) 硬件，加速AI训练、推理和高性能计算场景。DCU与NVIDIA GPU特性类似，支持大规模并行计算，但通常通过HIP C++编程接口进行开发

注：具体实现不限制于以上方式





(三) 赛题设置

基础题：智能矩阵乘法优化挑战——编译工具

1) g++ 编译和执行文件C++ 程序

编译: `g++ -o outputfile sourcefile.cpp`

执行: `./outputfile`

2) MPI和OpenMP编译和执行C++ 程序

编译: `mpic++ -fopenmp -o outputfile sourcefile.cpp`

执行: `./outputfile`

3) 曙光DCU编译和执行执行C++ 程序

执行: `hipcc source_dcu.cpp -o outputfile_dcu`

执行: `./outputfile_dcu`

注：也可以使用其他方式





(三) 赛题设置

基础题：智能矩阵乘法优化挑战——代码框架

1) 主要在CPU上优化的

```
// 初始化矩阵 (以一维数组形式表示, 用于随机填充浮点数据)
void init_matrix(std::vector<double>& mat, int rows, int cols) {
    std::mt19937 gen(42);
    std::uniform_real_distribution<double> dist(-100.0, 100.0);
    for (int i = 0; i < rows * cols; ++i)
        mat[i] = dist(gen);
}

// 验证计算优化后的矩阵计算和baseline实现是否结果一致, 可以设计其他验证方法, 来验证计算的正确性和性能
bool validate(const std::vector<double>& A, const std::vector<double>& B, int rows, int cols, double tol = 1e-6) {
    for (int i = 0; i < rows * cols; ++i)
        if (std::abs(A[i] - B[i]) > tol) return false;
    return true;
}

// 基础的矩阵乘法baseline实现 (使用一维数组)
void matmul_baseline(const std::vector<double>& A,
                    const std::vector<double>& B,
                    std::vector<double>& C, int N, int M, int P) {
    for (int i = 0; i < N; ++i)
        for (int j = 0; j < P; ++j) {
            C[i * P + j] = 0;
            for (int k = 0; k < M; ++k)
                C[i * P + j] += A[i * M + k] * B[k * P + j];
        }
}

// 方式1: 利用OpenMP进行多线程并发的编程 (主要修改函数)
void matmul_openmp(const std::vector<double>& A,
                  const std::vector<double>& B,
                  std::vector<double>& C, int N, int M, int P) {
    std::cout << "matmul_openmp methods..." << std::endl;
}

// 方式2: 利用子块并行思想, 进行缓存友好型的并行优化方法 (主要修改函数)
void matmul_block_tiling(const std::vector<double>& A,
                       const std::vector<double>& B,
                       std::vector<double>& C, int N, int M, int P, int block_size = 64) {
    std::cout << "matmul_block_tiling methods..." << std::endl;
}

// 方式3: 利用MPI消息传递, 实现多进程并行优化 (主要修改函数)
void matmul_mpi(int N, int M, int P) {
    std::cout << "matmul_mpi methods..." << std::endl;
}
```

```
// 方式4: 其他方式 (主要修改函数)
void matmul_other(const std::vector<double>& A,
                 const std::vector<double>& B,
                 std::vector<double>& C, int N, int M, int P) {
    std::cout << "Other methods..." << std::endl;
}

int main(int argc, char** argv) {
    const int N = 1024, M = 2048, P = 512;
    std::string mode = argc >= 2 ? argv[1] : "baseline";

    if (mode == "mpi") {
        MPI_Init(&argc, &argv);
        matmul_mpi(N, M, P);
        MPI_Finalize();
        return 0;
    }

    std::vector<double> A(N * M);
    std::vector<double> B(M * P);
    std::vector<double> C(N * P, 0);
    std::vector<double> C_ref(N * P, 0);

    init_matrix(A, N, M);
    init_matrix(B, M, P);
    matmul_baseline(A, B, C_ref, N, M, P);

    if (mode == "baseline") {
        std::cout << "[Baseline] Done.\n";
    } else if (mode == "openmp") {
        matmul_openmp(A, B, C, N, M, P);
        std::cout << "[OpenMP] Valid: " << validate(C, C_ref, N, P) << std::endl;
    } else if (mode == "block") {
        matmul_block_tiling(A, B, C, N, M, P);
        std::cout << "[Block Parallel] Valid: " << validate(C, C_ref, N, P) << std::endl;
    } else if (mode == "other") {
        matmul_other(A, B, C, N, M, P);
        std::cout << "[Other] Valid: " << validate(C, C_ref, N, P) << std::endl;
    } else {
        std::cerr << "Usage: ./main [baseline|openmp|block|mpi]" << std::endl;
    }

    // 需额外增加性能评测代码或其他工具进行评测
    return 0;
}
```

2) 主要在DCU上优化的

```
// 主要修改函数
__global__ void matmul_kernel(const double* A, const double* B, double* C, int n, int m, int p) {
    return;
}

void init_matrix(std::vector<double>& mat) {
    std::mt19937 gen(42);
    std::uniform_real_distribution<double> dist(-1.0, 1.0);
    for (auto& x : mat)
        x = dist(gen);
}

void matmul_cpu(const std::vector<double>& A, const std::vector<double>& B, std::vector<double>& C) {
    for (int i = 0; i < N; ++i)
        for (int j = 0; j < P; ++j) {
            double sum = 0.0;
            for (int k = 0; k < M; ++k)
                sum += A[i * M + k] * B[k * P + j];
            C[i * P + j] = sum;
        }
}

bool validate(const std::vector<double>& ref, const std::vector<double>& test) {
    for (size_t i = 0; i < ref.size(); ++i)
        if (std::abs(ref[i] - test[i]) > 1e-6)
            return false;
    return true;
}

int main() {
    std::vector<double> A(N * M), B(M * P), C(N * P), C_ref(N * P);
    init_matrix(A);
    init_matrix(B);

    // CPU baseline
    matmul_cpu(A, B, C_ref);

    // 主要修改部分
    // Allocate and copy to device, use matmul_kernel to compute in DCU
    double *d_A, *d_B, *d_C;

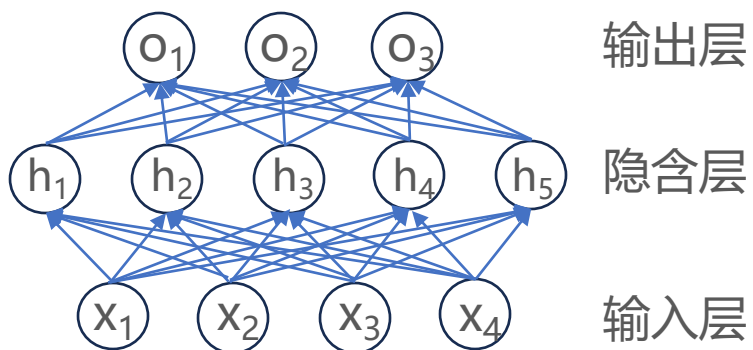
    // if (validate(C_ref, C)) {
    //     std::cout << "[HIP] Valid: 1" << std::endl;
    // } else {
    //     std::cout << "[HIP] Valid: 0" << std::endl;
    // }

    hipFree(d_A);
    hipFree(d_B);
    hipFree(d_C);
}
```

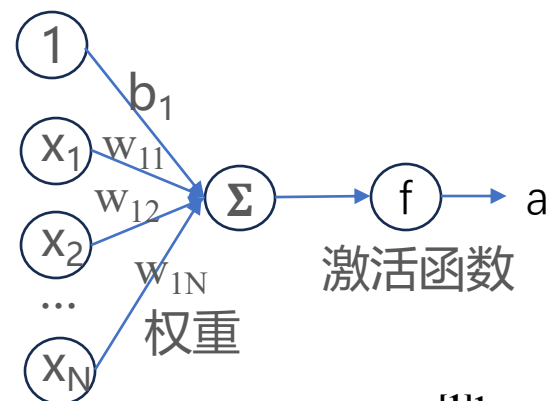



(三) 赛题设置

进阶题1：基于矩阵乘法的多层感知机(MLP)^[1,2]实现和性能优化挑战



MLP网络结构



神经元结构

^[1]<https://zh-v2.d2l.ai/>

^[2]<https://courses.d2l.ai/zh-v2/>

基于矩阵乘法，实现MLP神经网络计算，可进行前向传播、批处理，要求使用DCU加速卡，以及矩阵乘法优化方法，并进行全面评测；输入、权重矩阵由随机生成的浮点数组成：

- 输入层：一个大小为 $B \times I$ 的随机输入矩阵（ B 是 batch size=1024， I 是输入维度=10）；
- 隐藏层： $I \times H$ 的权重矩阵 W_1 + bias b_1 ，激活函数为 ReLU（ H 为隐含层神经元数量=20）；
- 输出层： $H \times O$ 的权重矩阵 W_2 + bias b_2 ，无激活函数，（ O 为输出层神经元数量=5）；



(三) 赛题设置

进阶题1：基于MLP¹实现和性能优化挑战——基本思路

以三层MLP为例：

假设：

- 输入层：`1024 × 10`
- 隐藏层：`10 × 20` (ReLU)
- 输出层：`20 × 5` (无激活)

变量定义：

- X 是输入矩阵 (Batch × 输入维度)
- W 是权重矩阵 (输入维度 × 输出维度)
- B 是偏置向量
- $@$ 表示矩阵乘法
- $\text{ReLU}(\cdot)$ 是非线性激活函数

1. 第一层前向传播

$$H_1 = \text{ReLU}(X @ W_1 + B_1)$$

$$X: [1024 \times 10]$$

$$W_1: [10 \times 20]$$

$$B_1: [1 \times 20]$$

$$H_1: [1024 \times 20]$$

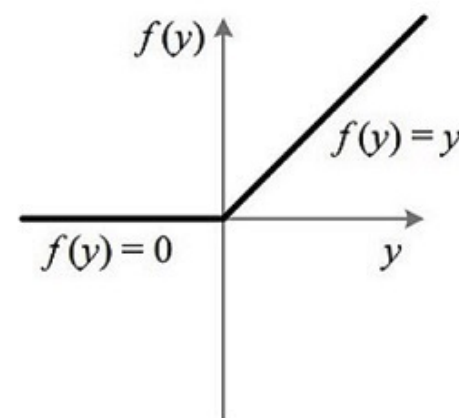
2. 第二层前向传播

$$Y = H_1 @ W_2 + B_2$$

$$W_2 = [20 \times 5]$$

$$B_2 = [1 \times 5]$$

$$Y = [1024 \times 5]$$



$$\text{ReLU}(x) = \begin{cases} x & \text{if } x > 0 \\ 0 & \text{if } x \leq 0 \end{cases}$$



(三) 赛题设置

进阶题1：基于MLP¹实现和性能优化挑战——代码框架

1. 内存初始化和数据准备

```
std::vector<double> h_X, h_W1, h_B1, h_W2, h_B2, h_H, h_Y;  
random_init(...);
```

2. 分配设备(DCU)内存并拷贝数据

```
hipMalloc(...); hipMemcpy(...);
```

3. 计算隐藏层输出: $H = \text{ReLU}(X \times W_1 + b_1)$

3.1. 执行矩阵乘法: $X \times W_1 \rightarrow H$

```
hipLaunchKernelGGL(matmul_kernel, ...);
```

//hipLaunchKernelGGL 是 HIP (Heterogeneous-computing Interface for Portability) 中用于在设备 (DCU) 上启动内核 (kernel) 函数的宏

3.2. 添加偏置和应用 ReLU 激活函数

```
hipLaunchKernelGGL(relu_kernel, ...)
```

4. 计算输出层结果: $Y = H \times W_2 + b_2$

```
hipLaunchKernelGGL(matmul_kernel, ...);
```

5. 输出结果复制回主机(CPU)和资源释放

```
hipMemcpy(h_Y.data(), d_Y, ...);  
hipFree(...);
```

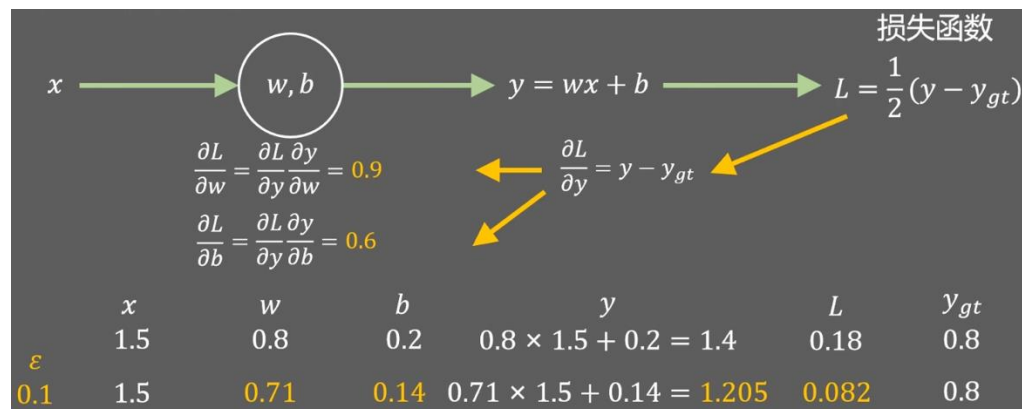
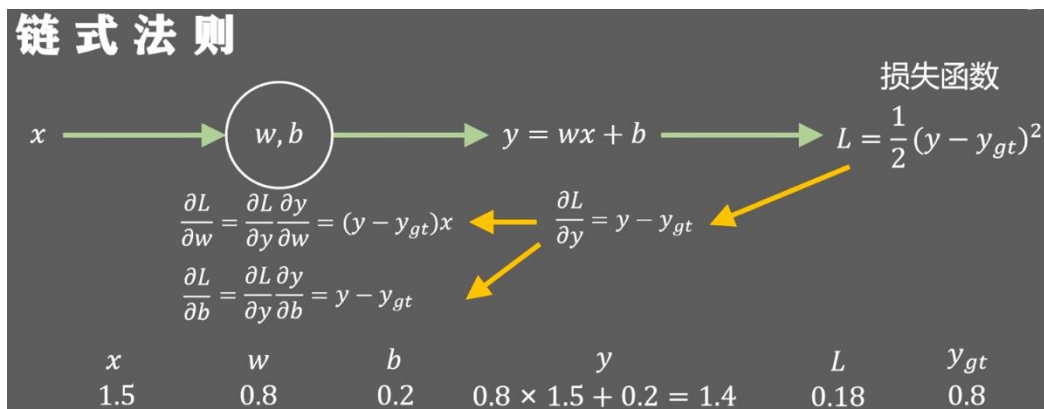




(三) 赛题设置

进阶题2：基于MLP的低轨卫星网络带宽预测性能优化挑战

链式法则



MLP反向传播更新 w, b 参数

完成MLP网络设计，要求能够进行前向传播，反向传播和通过梯度下降方法训练，并实现准确的LEO卫星网络下行带宽预测，需使用DCU加速卡，并对训练和推理性能进行全面评测：

- 输入：每次输入 $t_0, t_1, \dots, t_N$ 时刻的网络带宽值 ($N=10$) ；
- 输出：每次输出 t_{N+1} 时刻的网络带宽值；
- MLP：输入层、隐藏层、输出层神经元数量和层数，以及训练参数、损失函数可自行设计优化；
- 数据集：一维的带宽记录，每个数据对应一个时刻的带宽值（已上传到测试环境中）；

starlink_bw.json



(三) 赛题设置

进阶题2：基于MLP的低轨卫星网络带宽预测性能优化挑战——基本思路

第1步：数据预处理：将原始的一维带宽数据切分为训练样本，用于输入输出对的构造。

- 输入带宽样本格式为： $[t_0, t_1, \dots, t_9, t_{10}]$ ($N=10$)；输出样本为： t_{10+1}
- 滑动窗口构造多个样本；数据归一化（例如 min-max 或 Z-score）提升训练稳定性

第2步：MLP 网络结构与初始化：设计一个多层感知机（MLP）模型，指定每一层的结构和激活函数

- 输入带宽格式为： $[t_0, t_1, \dots, t_9, t_{10}]$ ($N=10$)；输出样本为： t_{10+1}

第3步：前向传播实现：初始化权重和偏置，完成输入到输出的传播，并记录每层的激活值供后续使用

第4步：反向传播和参数训练：实现从输出到输入的梯度计算，用于权重和偏置值的更新

第5步：性能评估与推理测试：在 DCU 上测试训练和推理（前向传播）的性能表现，并对精度做评估

注：准确的带宽预测往往需要调节网络参数，比如隐含层数量和大小、学习率、训练轮次、批大小等

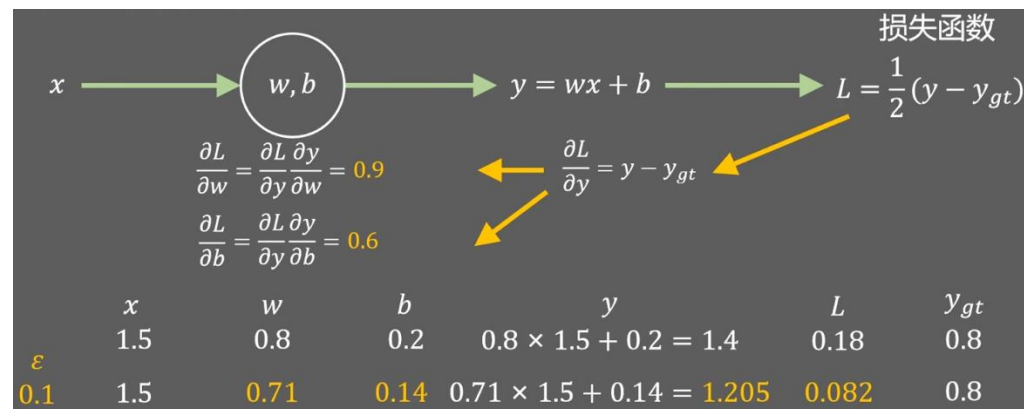
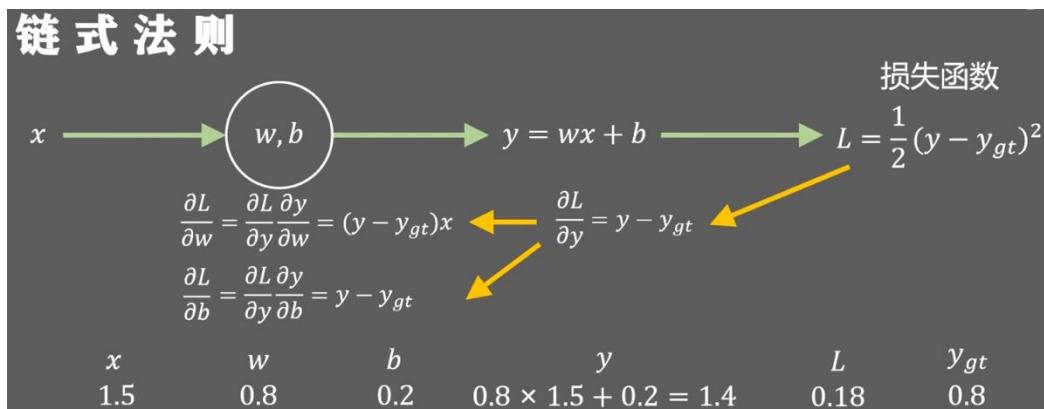




(三) 赛题设置

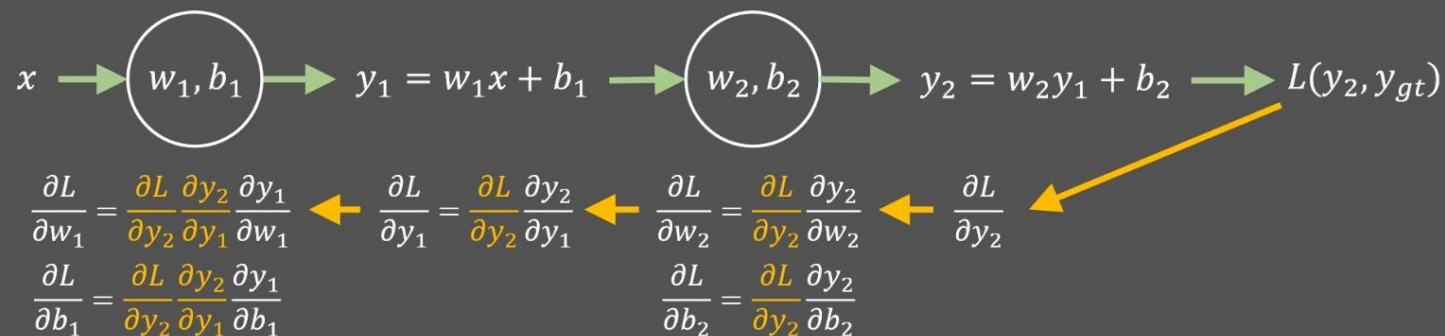
进阶题2：基于MLP的低轨卫星网络带宽预测性能优化挑战——反向传播

链式法则



MLP反向传播更新 w, b 参数

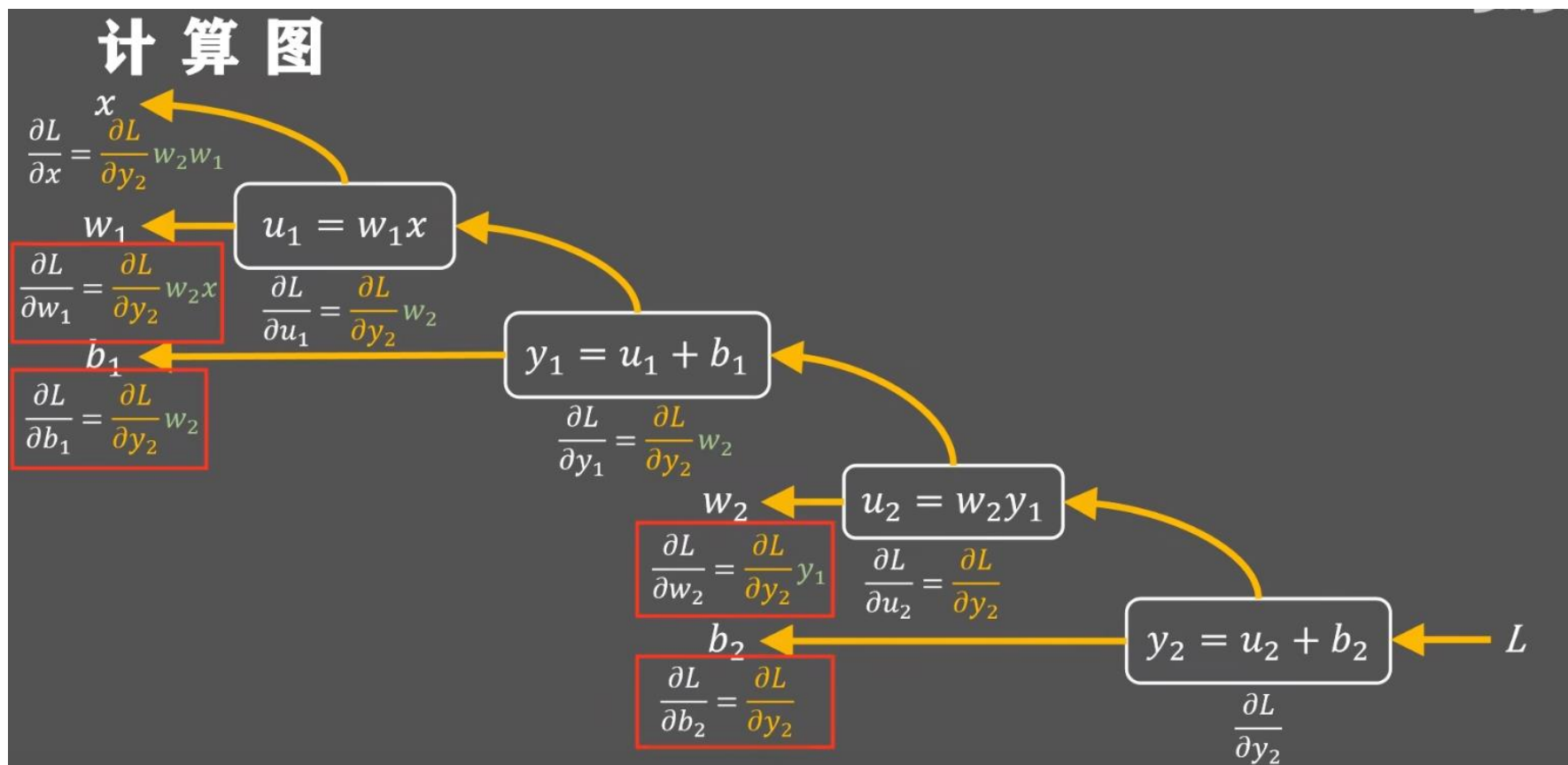
反向传播





(三) 赛题设置

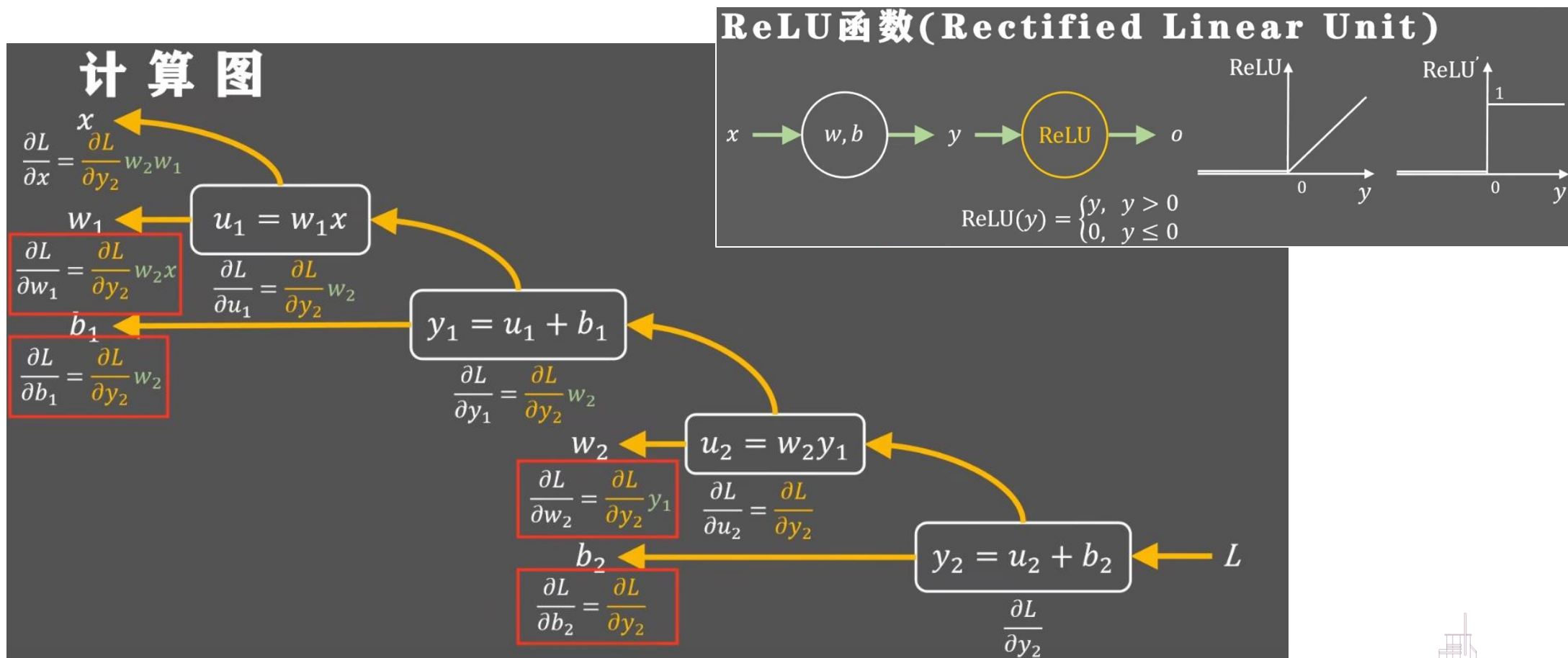
进阶题2：基于MLP的低轨卫星网络带宽预测性能优化挑战——反向传播





(三) 赛题设置

进阶题2：基于MLP的低轨卫星网络带宽预测性能优化挑战——反向传播





(三) 赛题设置

进阶题2：基于MLP的低轨卫星网络带宽预测性能优化挑战——反向传播

示例：

网络结构参数：

输入维度：4

第一隐藏层神经元：3

第二隐藏层神经元：2

输出层神经元：1

批大小：5

激活函数：ReLU

正向传播过程

$$\begin{aligned} Z_1 &= XW_1 + b_1 && \rightarrow \text{shape } (5 \times 3) \\ A_1 &= \text{ReLU}(Z_1) && \rightarrow \text{shape } (5 \times 3) \\ Z_2 &= A_1W_2 + b_2 && \rightarrow \text{shape } (5 \times 2) \\ A_2 &= \text{ReLU}(Z_2) && \rightarrow \text{shape } (5 \times 2) \\ Z_3 &= A_2W_3 + b_3 && \rightarrow \text{shape } (5 \times 1) \\ \hat{y} &= Z_3 && \rightarrow \text{最终输出} \end{aligned}$$

假设损失函数是均方误差 (MSE)：

$$L = \frac{1}{2} \|\hat{y} - y\|^2$$

第一层：

$$\delta_1 = (\delta_2 W_2^T) \circ \text{ReLU}'(Z_1) \quad (5 \times 3)$$

$$\begin{aligned} \frac{\partial L}{\partial W_1} &= X^T \delta_1 \quad (4 \times 3) \\ \frac{\partial L}{\partial b_1} &= \text{sum}(\delta_1) \quad (1 \times 3) \end{aligned}$$

第二层：

$$\delta_2 = (\delta_3 W_3^T) \circ \text{ReLU}'(Z_2) \quad (5 \times 2)$$

其中：

- ReLU'(Z₂): ReLU mask, 为 0/1 组成的矩阵
- : 逐元素乘 (Hadamard 乘积)

梯度：

$$\begin{aligned} \frac{\partial L}{\partial W_2} &= A_1^T \delta_2 \quad (3 \times 2) \\ \frac{\partial L}{\partial b_2} &= \text{sum}(\delta_2) \quad (1 \times 2) \end{aligned}$$

第三层 (输出层)：

$$\delta_3 = \frac{\partial L}{\partial Z_3} = \hat{y} - y \quad (5 \times 1)$$

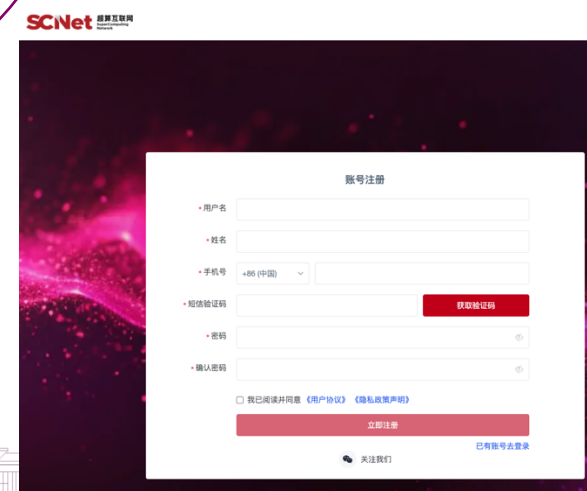
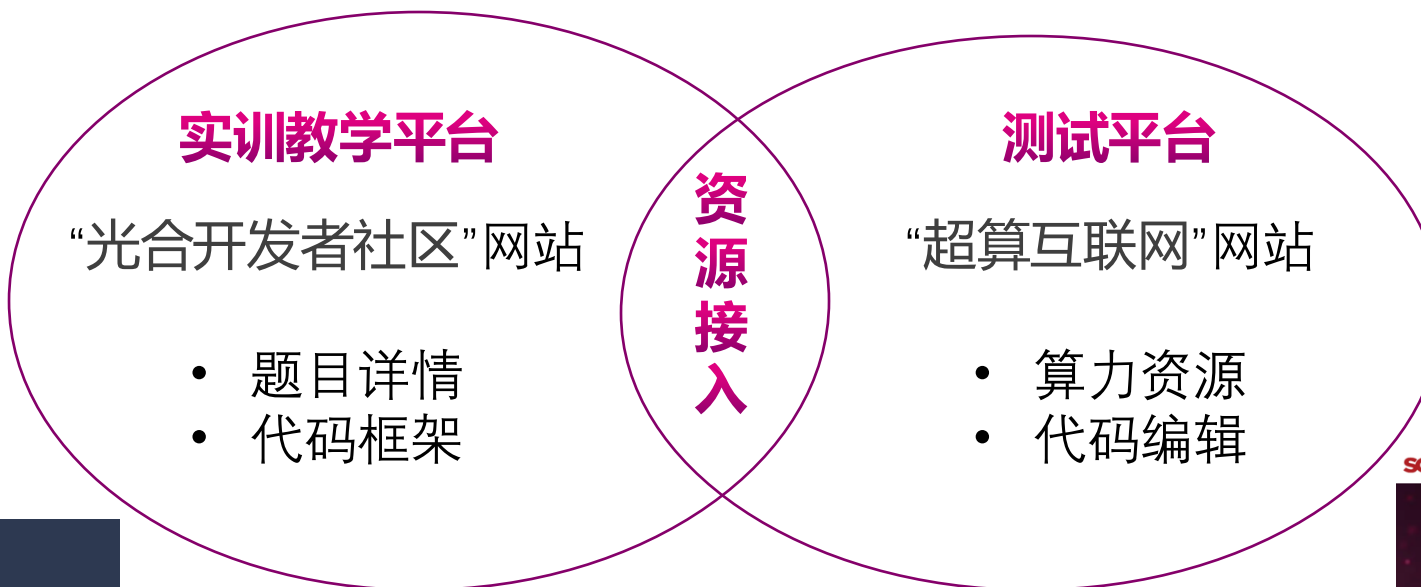
梯度：

$$\begin{aligned} \frac{\partial L}{\partial W_3} &= A_2^T \delta_3 \quad (2 \times 1) \\ \frac{\partial L}{\partial b_3} &= \text{sum over batch}(\delta_3) \quad (1 \times 1) \end{aligned}$$

一般取批次的均值后，
计算下降的梯度



(四) 算力资源



预祝大家比赛顺利！

